SpendWise: Modern MERN Expense Tracker

Full Stack Frameworks Mini Project Evaluation

Akshar Sakhi

Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham

Evaluation Date: 16 October 2026

Repository: github.com/aksharsakhi/mern-expense-tracker

Problem Statement & Motivation

The Challenge

- Personal financial records are fragmented across static notes, paper bills, and disjointed bank statements.
- Lack of real-time insights into monthly category distributions (e.g., dining, housing, transportation).
- Absence of instant metrics for true monthly net balance and dynamic savings velocity.

The Solution: SpendWise

- Centralized full-stack web application built on the modern **MERN** stack.
- Fast transaction logging with payment methods, category tagging, and date tracking.
- High-speed aggregation pipelines calculating cashflow statistics with visual Chart.js dashboards.

System Architecture & Technology Stack

Three-Tier Architectural Overview

- **Presentation Tier (Client):** React 18, Vite, Chart.js, Lucide Icons, Glassmorphism CSS design system.
- **Application Tier (API):** Node.js & Express.js RESTful API, CORS, centralized validation, and error middleware.
- **Data Tier (Database):** MongoDB 8.x with Mongoose ODM, indexed collection queries, and multi-stage aggregation.

Frontend (Vite)

- Component modularity
- Responsive dashboard
- Dark/Light mode theme

Backend (Express)

- Clean route separation
- Schema validation
- Dynamic month query

Database (MongoDB)

- Strict Mongoose model
- Multi-facet aggregations
- Compound indexing

RESTful API Design & Data Contracts

Method	Endpoint	Functionality
GET	/api/transactions	Paginated & filtered records (?month=2026-10)
GET	/api/transactions/summary	Aggregation totals, categories, daily trends
POST	/api/transactions	Create new transaction with input validation
GET	/api/transactions/:id	Retrieve transaction details by ObjectId
PUT	/api/transactions/:id	Update existing transaction attributes
DELETE	/api/transactions/:id	Remove transaction record
POST	/api/transactions/seed	One-click demonstration dataset seeding

JSON Payload Schema

```
{ "amount": 18500, "type": "expense", "category": "Housing", "date": "2026-10-02", "paymentMethod": "Net Banking", "note": "Apartment Rent" }
```

MongoDB Aggregation Pipelines

Multi-Faceted Pipeline Execution

A single round-trip database query utilizing `$facet`:

1. **Total Balances:** `$group` by `$type` to sum total income and total expense.
2. **Category Distribution:** Filter expenses and compute sub-totals and percentages per category.
3. **Daily Cashflow Velocity:** Group by ISO date string to feed daily spending charts.

Computed Metrics Returned

- **Net Balance:** Total Income $-$ Total Expense
- **Savings Rate:** $\frac{\text{Income} - \text{Expense}}{\text{Income}} \times 100\%$
- **Category Share:** $\frac{\text{Category Total}}{\text{Total Expense}} \times 100\%$
- **Payment Method Split:** Breakdown across UPI, Cards, and Net Banking.

Production Deployment & DevOps Architecture

Containerization (`Docker`)

- **Backend:** Multi-stage Node.js Alpine container with non-root security.
- **Frontend:** Multi-stage build producing static bundle served through an optimized **Nginx** Alpine reverse proxy.
- **Compose:** `docker-compose.yml` orchestrating MongoDB, Express API, and Nginx Client.

Cloud Infrastructure as Code

- **Render Blueprint (`render.yaml`):** Automated cloud deployment for Express web service and React static site.
- **Vercel (`vercel.json`):** Client-side SPA routing and transparent API proxying.
- **Hybrid Production Serving:** Express dynamically serves `client/dist` in production single-instance environments.

Evaluation Demo Highlights & Key Learnings

Live Demonstration Highlights

- Full CRUD workflow with responsive modal forms and toast feedback.
- Interactive category donut and daily cashflow bar charts.
- Dynamic month selector with instantaneous metric updates.
- One-click CSV transaction export.

Key Takeaways

- End-to-end full-stack development using modern MERN principles.
- Scalable database schema design and aggregation optimization.
- Industrial DevOps practices: Docker containerization and cloud deployment configs.

Thank You! Questions & Demonstration